

DD2459 Software Reliability 2025
Take-home Exam
Tuesday 11 March, 15.00 – Thursday 13 March, 12.00 midday
(Version 1.0)

Instructions.

Please read the following instructions carefully.

- (1) Only hard-copy exam submissions are accepted. Clearly mark at the top of each sheet of paper you use: (a) your name, (b) the page number.
- (2) On your front page indicate: (a) how many pages are contained in your work in total, (b) your name (c) your personal or KTH number, (d) your e-mail address (in case I need to contact you)
- (3) There are two ways to **submit** your work. (a) Your work may be **handed in personally by you only** to EECS studentexpedition, E building, level 4, Lindstedsvägen 3, no later than Thursday 13th March 2025 at 12.00 midday. After this time, it will be marked as late, and marks will be subtracted. **Please take identification with you if you submit this way, e.g. a legitimization or passport.** (b) If you are unable to reach the studentexpedition yourself (for example if you are at work) you may **post** your manuscript to: Karl Meinke, EECS School, KTH Kungliga Tekniska Högskolan, 100 44 Stockholm, Sweden. The date on the postmark will be taken as the date of your submission. The deadline of March 13th, 2025 applies to all manuscripts submitted by post.
- (4) If manuscripts are submitted in any other place or by any other means than those described in Part (3) then the examiner and EECS School cannot be held responsible in the case that manuscripts are lost. In the case of postal submission, it is strongly recommended that you keep a digital copy or photocopy in case of postal loss. KTH cannot be held responsible for loss in any national postal system.
- (5) If you have any questions about the exam (for example, if you do not understand a question) you may call the examiner on 08 790 6337. Please do not call before 10.00 am or after 5.00pm, and be aware that I may be in a meeting! You can also e-mail the examiner at karlm@kth.se.
- (6) I will publish any typographical errors and corrections that become known during the exam on the course exam web page and notify you with Canvas. Therefore, you should check back regularly on the course exam page for updates that will have new version numbers 1.x.
- (7) You may use your course notes, books and the internet. However, all material you submit must be your own. (a) You are not allowed to discuss your answers with anyone else until the exam is finished. (b) You are not allowed to copy anyone else's work. (c) By handing in your manuscript you are declaring that you have read and abide by all the rules on this cover page. (d) In the case that cheating is suspected, disciplinary action will be taken.
- (8) Hand-written answers must be written clearly. No marks will be awarded for work that I cannot read. You can write your answers in Swedish or in English. You can submit a hand-written manuscript or a typed manuscript or any combination of both.

NOTE: For a full score of 100 points you should answer all 4 questions below.

Question 1. (Total 35 points)

Write appropriate *pre-* and *post-conditions* for the following mathematical operations using the JML specification language (i.e. write *requires-ensures* conditions).

You should try to avoid both over and under-specification, i.e. write all and only the constraints necessary for each pre- and post condition. Do not omit relevant constraints or add irrelevant ones. **Hint: if you write a text comment next to each formula explaining its intended meaning then your work will be clearer and you will be less likely to make mistakes.**

If you are unsure how any mathematical operation/method is defined you may look it up online or in a book. You do not have to provide Java code for any of the operations, only JML.

Recall (or look up) that a *max-heap* is a labelled tree data structure that satisfies the following property. In a *max-heap*, for any given node c , if node p is the parent node of c (i.e. c is a child node of p) then the *key* (i.e. the value stored) at p is greater than or equal to the *key* at c . Note that a max-heap is not necessarily a complete tree, i.e. it can be unbalanced. The node at the top of the max-heap (with no parents) is called the *root* node. **Note:** the empty heap, with no keys at all, will be regarded as a valid max-heap.

In all questions below, we will assume that a *key* is a Java `Integer` object.

A common implementation of the max-heap uses a binary heap in which the tree is a binary tree. This can be simply and easily implemented using a linear array, in which each array element represents one node in the binary tree, and stores one element (the node key) of type `Integer`. The parent-child relationship is implicitly represented by the indexing of the array, so each node at index i has its parent node at index $\lfloor (i-1)/2 \rfloor$ (Here $\lfloor . \rfloor$ is the so-called “floor function”).

In all questions below, we will assume that a max-heap is implemented as a binary heap. **You should reuse your answer to part (i) to make your answers to parts (ii) and (iii) shorter.**

(i) **(10 points)** The *max-heap property* for an *Integer* array object `inArray`

```
boolean isMaxHeap(Integer [] inArray)
```

Input: An `Integer` array `inArray`.

Output: `true` if, and only if, `inArray` represents a valid max-heap.

Model Answer

```
requires
// inArray exists
inArray != null
```

```
ensures
    \result == true <=>

    // result is true iff, every well-defined node (except
    // root node) is a child of a well-defined parent node
    // and the key of this child is not larger than the
    // key of its parent

    \forallall (int i; 0 < i < \old(inArray).length;
    \old(inArray)[i] != null =>
        ( \old(inArray)[Math.floor((i-1)/2)] != null &&
        \old(inArray)[i] <=
            \old(inArray)[Math.floor((i-1)/2)] )
    )
```

Note: it could be that:

```
(\old(inArray)[i]).intValue() <=
    (\old(inArray)[Math.floor((i-1)/2)]).intValue()
```

is more correct in the above. Java documentation seems a little unclear on this point IMO. I would give the same marks for either approach.

(ii) **(5 points)** A `getRoot` function

```
Integer getRoot(Integer [] inArray)
```

Input: An `Integer` array `inArray` which is a valid max-heap.

Output: The `Integer` object which is the key at the root of the max-heap `inArray`.

Model Answer

```
requires
```

```
// inArray exists and is a valid max heap,
inArray != null & isMaxHeap(inArray) == true &
// and root node exists
inArray.length > 0 & inArray[0] != null
```

```
ensures
// original root key is returned
\result == \old(inArray)[0]
```

(iii) **(20 points)** A deleteRoot function

```
Integer [] deleteRoot(Integer [] inArray)
```

Input: An Integer array inArray which is a valid max-heap.

Output: A valid max-heap which is obtained from inArray by deleting the key at the root node.

Model Answer

Note: a 0 or 1 length array must be a max-heap.

requires

```
// inArray exists and has 0 elements (so 0 keys, i.e.  
empty heap)
```

```
inArray != null & inArray.length == 0
```

ensures

```
// \result exists but has no elements
```

```
\result != null & \result.length == 0
```

also

requires

```
// inArray exists and has one element (so 0 or 1 keys)
```

```
inArray != null & inArray.length == 1
```

ensures

```
// \result exists, has 1 element but no keys (so 0 keys)
```

```
\result != null & \result.length == 1 &
```

```
\result [0] == null
```

also

requires

```
// inArray exists and has more than one element (so 0 or  
more keys) and is a max-heap
```

```
inArray != null & inArray.length > 1 &&
```

```
isMaxHeap(inArray) == true
```

ensures

```
// \result exists, with same number of elements
```

```
\result != null & (\old(inArray)[0].length ==
```

```
\old(inArray).length
```

&

```
// if inArray had no root key, then since it is a max-
```

```
// heap, it had no keys at all, so \result has no keys
```

```
// either
```

```
\old(inArray)[0] == null =>
```

```

forall (int i; 0 < i < \result.length;
\result [i] == null)

&
\old(inArray)[0] != null =>
// if inArray had a root key, then \result is a max-heap,
// in which the number of keys with value
// \old(inArray)[0] should be 1 less than \old(inArray).

(
\num_of(int j; 0 < j < \old(inArray).length;
\old(inArray)[j].equals(\old(inArray)[0])) )
==
\num_of(int j; 0 <= j < \result.length;
\result[j].equals(\old(inArray)[0])) )
)
&&
// also the number of keys of every value *other
// than* \old(inArray)[0], (i.e. the old root key value)
// is the same in \result as it originally was in
// \old(inArray)

(
forall (int i; 0 < i < \old(inArray).length &&
!\old(inArray)[i].equals(\old(inArray)[0]));

\num_of(int j; 0 < j < \old(inArray).length;
\old(inArray)[j].equals(\old(inArray)[i])) )
==
\num_of(int j; 0 <= j < \result.length;
\result[j].equals(\old(inArray)[i])) )
)

```

Question 2. (Total 10 points)

- (i) (4 points) Write down four different metamorphic equations that should be satisfied by any test case for the *root mean square function (aka. RMS)*:

```
float rms(float[] sample)
```

Input: A one-dimensional array `sample` of `float` (the data set).

Output: The `float` result which should be the square root of the arithmetic mean of the squares of all `sample` values `sample[0] ,..., sample[n]`.

Note: For this example, you may assume that floating point operations are perfectly accurate.

Clearly define the data transformation

T: `float[]` → `float[]`

used on the input array for each of your four different metamorphic equations.

Model Answer: The rms function satisfies any permutative law (similar to the inclusive law mentioned in Lecture 6) such as

$\text{rms}([x_1, \dots, x_n]) = \text{rms}(\text{perm}([x_1, \dots, x_n]))$ where $\text{perm} : R^* \rightarrow R^*$ is any permutation on lists of floats, for example:

1. $\text{perm}([x_1, \dots, x_n]) = [x_n, \dots, x_1]$ is the reversal permutation
2. $\text{perm}([x_1, \dots, x_n]) = [x_n, x_1, \dots, x_{n-1}]$ is the shift-1 right permutation

The rms function also satisfies negation laws such as

$\text{rms}([x_1, \dots, x_n]) = \text{rms}(\text{negative}([x_1, \dots, x_n]))$ where $\text{negative}()$ is a negative transformation, for example:

3. $\text{negative}([x_1, \dots, x_n]) = [-x_1, \dots, -x_n]$
4. $\text{negative}([x_1, \dots, x_n]) = [-x_1, x_2, \dots, x_n]$

You may be able to find other laws that hold?

- (ii) (6 points) For each of your four metamorphic equations $e_i, i = 1, \dots, 4$, derived for Part (i), consider its associated transformation T_i . Starting from one seed test case

$x_0 : \text{float}[]$

discuss whether the sequence of test cases

$$x_{j+1} = T_i(x_j) : j = 0, 1, 2, \dots$$

generated by iterating T_i repeats itself.

If all of your test case sequences (x_i) above are finite, does there exist a metamorphic equation e for `SetEquality` with a transformer T which can generate a test suite of $\text{TS} = \{ (x_0, y_0), \dots, (x_k, y_k) = T^k(x_0, y_0) \}$ of unlimited size by simply iterating the transformer T k -times?

Model Answer: Given a seed test case such as $[1, \dots, n]$ a new test case with reversal would be

$$T([1, \dots, n]) = [n, \dots, 1].$$

However, if we try to apply the reversal permutation T twice then we get

$$T^2([1, \dots, n]) = [1, \dots, n].$$

So higher powers of reversal $T^n([1, \dots, n])$ are just one of these two vectors and we only get two distinct test cases.

Notice that `shift-1 right` gives n new test cases and in general all other permutations give different numbers of test cases, but no fixed permutation gives more than n . Note also

that some permutations can give no new tests, depending also on the seed test, e.g. reversal of a symmetric vector!

Looking at negations, we can see that in each case

$\text{negative}(\text{negative}([x_1, \dots, x_n])) = [x_1, \dots, x_n]$

So again higher powers of negation $\text{negative}^n([x_1, \dots, x_n])$ are just repetitions one of these two vectors and we only get two distinct test cases.

Question 3. (Total 21 points)

Consider the UML state machine diagram for a *vehicle cruise controller* given in Figure 1 below.

A cruise controller is a real-time embedded application that interacts with the gas (`acc`) and brake (`brake`) pedal events, as well as the dashboard (`off`) and (`resume`) command events, to set and maintain a constant cruising speed (`desiredSpeed`), based on the vehicles current speed (`speed`). The application also interacts with global vehicle signals such as switching the engine on and off. All events, Boolean and floating-point variables in the diagram below should be clear, from this application description. You may assume that every action `Increase(speed)` will increase the value of `speed` by a fixed constant `c` by the end of one time-unit, i.e.

$\text{always}(\text{speed} == y \ \& \ \text{Increase}(\text{speed}) \Rightarrow \text{next}(\text{speed} \geq y+c))$

While every action `Monitor(speed)` updates the current value of `speed` according to the actual physical speed of the vehicle (given by some sensor such as wheel rotation) after one time-unit.

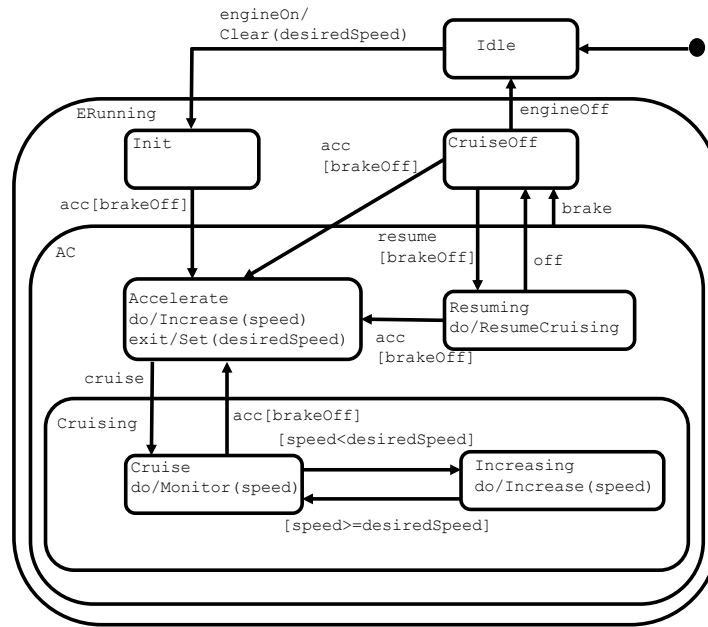


Figure 1. UML state machine for a cruise controller

- (i) **(15 points)** For each of the LTL formulas below, state whether it is true or false about the cruise controller model in Figure 1. If a formula is false, you should give a counterexample trace that demonstrates an execution which fails the property.

a) $\text{always}(\text{state} = \text{ERunning.CruiseOff})$

Model Answer False, e.g. counterexample trace = engineOn

b) $\text{always}(\text{state} = \text{ERunning.AC} \Rightarrow$
 $\text{state} = \text{ERunning.AC.Accelerate} \mid$
 $\text{state} = \text{ERunning.AC.Resuming} \mid$
 $\text{state} = \text{ERunning.AC.Cruising})$

Model Answer True – this is the meaning of the “or” state ERunning.AC

c) $\text{sometime}(\text{state} = \text{ERunning.AC.Cruising})$

Model Answer False, we can be in a manual non-cruising mode indefinitely (as one would expect)

e.g. counterexample trace = engineOn,acc,brake,acc,brake,acc,brake,...
or (as a regular expression) trace = engineOn,(acc,brake)*

d) $\text{always}(\text{state} = \text{ERunning.AC.Cruising.Increasing} \ \& \ \text{speed} \leq \text{desiredSpeed} \Rightarrow$
 $\text{next}(\text{state} = \text{ERunning.AC.Cruising.Increasing}))$

Model Answer False, a counterexample can be built as follows. Suppose that
 $\text{desiredSpeed} - \text{speed}_s = k > 0$

at time s on entering state

ERunning.AC.Cruising.Increasing

from state

ERunning.AC.Cruising.Cruise

after the input

`trace = engineOn,acc,cruise`

and let $t = k/c$. Then after t time units:

$speed_{s+t} = speed_s + (k/c * c) = speed_s + k = desiredSpeed$

Then according to the diagram in Figure 1, at this time the next state is

`ERunning.AC.Cruising.Cruise`

as the transition is enabled.

- e) `always(state = ERunning.AC.Cruising.Increasing & brake $\Rightarrow$
next(state=CruiseOff))`

Model Answer True – this is the meaning of an exit transition from an “or” state

- f) `always(state = ERunning.AC.Cruising.Cruise & speed > desiredSpeed $\Rightarrow$
next(speed <= desiredSpeed))`

Model Answer False, a counterexample can be built as follows. Suppose that

$desiredSpeed < speed_s$

at some time s in state

`ERunning.AC.Cruising.Cruise`

after the input

`trace = engineOn,acc,cruise`

There is no transition enabled, and so after one execution of `Monitor(speed)`, the next state would be $speed_{s+1}$. Now for example, if the vehicle is being accelerated down a hill by gravity, then $speed_{s+1} > speed_s > desiredSpeed$. Thus, `next(speed <= desiredSpeed)` must be false.

- (ii) **(6 points)** A well-known safety hazard with cruise controller software is popularly known as *unintended acceleration*, when the vehicle can accelerate in an uncontrollable and dangerous way.
- a) Explain, how you could use software testing to uncover such an implementation error in a code implementation of the UML state machine diagram of Figure 1 above.
Model Answer The design given in Figure 1 may have the problem of unintended acceleration when the vehicle descends a hill. At that time, in the cruising state, the vehicle can accelerate above the set speed, and nothing in the design will reduce the speed. In a real implementation, we could test the cruise controller with any test input that covers the cruising state node while descending a hill.
- b) Comment on whether model-based testing using Figure 1 would generate such a test case.
Model Answer Model-based testing could be used to generate a test case that covers node cruising. However, unless the test is conducted while descending a hill, the unintended acceleration error could not be found. On a flat road, or while ascending a hill, the forces of gravity and friction are decelerating on the vehicle, and unintended acceleration will not be seen. So the model-based strategy must be extended with changes to environmental variables such as road choice.
- c) Based on your answer to (b), what conclusions can you draw about model-based

testing of embedded software?

Model Answer In the development of embedded software, environmental variables must be taken into account, even where these do not appear in a model. Suitable values must also be assigned to the environmental variables, and not just the system variables.

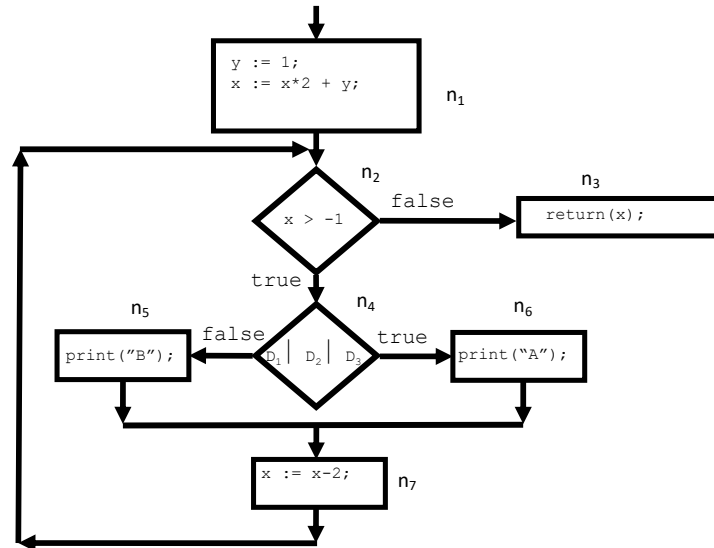
Question 4. (Total 34 points)

Harry Hopeful wanted to rigorously unit test the following code

```
1. myCode(int x): int
2. {
3.     int y;
4.     x:= x*2 + y;
5.     while ( x>-1 ) do
6.     {
7.         if ((x mod 2 == 0) & x > 1 & x <= 2 |
8.             (x <= 2 & !( x mod 2 == 0) & !(x>1) |
9.             !(x>1) & (x mod 2 == 0) & x <= 2)
10.        then
11.            print("A");
12.        else
13.            print("B");
14.        x := x - 2;
15.    } ;
16.    return x;
17. }
```

(a) (2 points) Draw a condensation graph for the routine myCode.

Model Answer



In the above graph, there are 3 disjuncts:

$D_1 = ((x \bmod 2 == 0) \ \& \ x > 1 \ \& \ x \leq 2$

$D_2 = (x \leq 2) \ \& \ !(x \bmod 2 == 0) \ \& \ !(x > 1)$

$D_3 = !(x > 1) \ \& \ (x \bmod 2 == 0) \ \& \ x \leq 2$

- (b) (8 points) Write out a full set of test requirements for the condensation graph of part (a) using the logic coverage model of *correlated active clause coverage* (CACC) (also known as *Masking MCDC*).

Model Answer

There are 2 predicates in the routine `myCode`, namely
 $(x > -1)$

which has 1 clause, and the predicate

$D_1 \mid D_2 \mid D_3$

consisting of 3 disjuncts, each of which is a conjunction of 3 (negated or unnegated) clauses:

$A = (x \bmod 2 == 0)$,

$B = x > 1$,

$C = x \leq 2$

So now we have identified all the clauses in `myCode`.

Notice that there is some interaction between these clauses, since $!B \rightarrow C$.

For the first predicate with just 1 clause CACC is simple, as this single clause must determine the whole predicate:

CACC TR1.(a) reach n_2 and $x > -1$

CACC TR1.(b) reach n_2 and $!x > -1$

For the second predicate P with 3 clauses we must write out its truth table with 2^3 rows.

Row	$A = (x \bmod 2 == 0)$	$B = (x > 1)$	$C = (x \leq 2)$	P	Comment
1	0	0	0	0	
2	0	0	1	1	disjunct D_2
3	0	1	0	0	
4	0	1	1	0	
5	1	0	0	0	
6	1	0	1	1	disjunct D_3
7	1	1	0	0	
8	1	1	1	1	disjunct D_1

We must now identify which clauses determine P using this truth table.

We see that A determines P with $B = 1$ and $C = 1$ (and only this assignment), so

CACC TR2.(a) reach n_4 and A

CACC TR2.(b) reach n_4 and $\neg A$

We see that B determines P with $A = 0$ and $C = 1$ (and only this assignment), so

CACC TR3.(a) reach n_4 and B

CACC TR3.(b) reach n_4 and $\neg B$

Finally, we see that C determines P with $A = 0$ and $B = 0$ (also $A=1$ and $B=0$, and $A=1$ and $B=1$), so

CACC TR4.(a) reach n_4 and C

CACC TR4.(b) reach n_4 and $\neg C$

Note that we should use the above side conditions on the non-determining clauses for test cases if we can. We investigate if this is possible in the next question.

- (c) (18 points) Write out a full set of test cases to satisfy as many of your CACC requirements as possible. What is the maximum percentage of coverage that can be achieved for CACC requirements? What can you say about the masking effects of the program clauses on each other?

Model Answer

CACC TC1.(a) $x = 0$ reaches n_2 and makes $x > -1$ true. The predicate is set to true.

CACC TC1.(b) $x = -1$ reaches n_2 and makes $x > -1$ false. The predicate is set to false.

When A determines P

CACC TC2.(a) here we see that whenever we reach n_4 then x is always an odd number (why?) and so $A = (x \bmod 2 == 0)$ is false. So, there is no satisfying test case for CACC TR2(a).

CACC TC.(b) As above, we see that whenever we reach n_4 then x is always an odd number (why?) and so $!A \equiv (x \bmod 2 \neq 0)$ is true. However, if x is odd then B and C cannot both be true, and our original and only assignment $B = 1$ and $C = 1$, when A determines P cannot be achieved.

However, we can choose $x = 0$ which reaches n_4 and makes $!A$ true (but B false and C true). The predicate is then set to true. This does not violate the CACC coverage definition.

When B determines P

CACC TC3.(b) For any input value $x \geq 1$, the execution on x reaches n_4 and makes B true (also A false and C false). The predicate is set to false. This does not violate CACC as the predicate is toggled with this choice.

CACC TC3.(b) For $x = 0$, the execution reaches n_4 and makes $!B$ true. (Also A false and C true). The predicate is set to true.

When C determines P

CACC TR4.(a) For $x = 0$, the execution reaches n_4 and makes C true (Also A false and B false). The predicate is set to true.

CACC TR4.(b) For $x = 1$, the execution reaches n_4 and makes $!C$ true (Also A false and B true). The predicate is set to false.

This test suite covers 7 out of 8 test requirements, i.e. at most 87.5% CACC coverage is possible.

The clause $x > -1$ is not at all masked, and it can't be as there is no other clause to mask it.

However, for clauses A , B and C we can never keep the same assignment α and toggle the determining clause only to achieve the necessary predicate toggling. So, each of the clauses A, B, C is masked by the others to some extent, as well as by $x > -1$.

- (d) (6 points) Choose any one test requirement that cannot be satisfied in part (c) and using the method of symbolic evaluation write a constraint table which shows that the requirement cannot be satisfied.

Model Answer

As we saw in part (c), there is no satisfying test case for CACC TR2(a), i.e. reach n_4 and A . Let us build a constraint table for a simple path to n_4 which demonstrates this.

Node	Instruction	Constraint
1	$y := 1$	$y' = 1$
1	$x := x*2 + y$	$x' = x*2 + 1$
2	$x > -1$	$x*2 + 1 > -1$
4	$D_1 \mid D_2 \mid D_3$	$A \equiv (x' \bmod 2 \neq 0)$

Then to satisfy CACC TR2(a), we have

$$x^2 + 1 > -1 \ \& \ (x' \bmod 2 == 0)$$

That is

$$x^2 + 1 > -1 \ \& \ (x^2 + 1 \bmod 2 == 0)$$

But

$$x^2 + 1 \bmod 2 == 1$$

for any input value of x , which establishes that CACC TR2(a) is unsatisfiable. Notice that when we leave n_7 then again

$$x^2 + 1 \bmod 2 == 1$$

So, no other path to n_4 will satisfy CACC TR2(a) either.